

Spring Boot

비동기

비동기통신(axios 사용)

비동기통신 | axios를 이용한 서버와의 데이터 통신 - 1

axios 라이브러리를 사용하면 서버와 데이터를 통신할 수 있다.

데이터 통신이란 데이터를 주고 받는다는 의미다.

우리는 서버로 spring을 사용하기 때문에 axios를 이용하면 서버인 spring과 화면단인 react가 서로서로 데이터를 주고 받을 수 있다.

react에서 spring 서버로 통신 명령어를 보내는 것을 요청(request), 이 요청에 대한 결과를 spring이 react로 보내주는 것을 응답(response)라 한다.

axios 라이브러리를 사용하기 위해 먼저 HTML의 BOBY 태그 마지막에

<script src="https://cdn.jsdelivr.net/npm/axios@1.13.2/dist/axios.min.js"></script> 코드를 추가한다.

이 코드는 다른 script 파일보다 윗 줄에 추가되어야 한다.

axios 활용 서버와의 통신 문법

데이터 통신은 Restful API 문법을 활용한다. [] 안에 작성한 내용은 생략 가능한 내용임을 의미한다.

- 서버의 데이터를 react에서 받기

```
axios.get(url, [설정내용]).then(통신 성공 후 실행 내용).catch(실행 실패 시 실행 내용);
```

```
axios.delete(url, [설정내용]).then(통신 성공 후 실행 내용).catch(실행 실패 시 실행 내용);
```

- react의 데이터를 서버에 전달하기

```
axios.post(url, [전달할 데이터], [설정내용]).then(통신 성공 후 실행 내용).catch(통신 실패 시 실행 내용);
```

```
axios.put(url, [전달할 데이터], [설정내용]).then(통신 성공 후 실행 내용).catch(통신 실패 시 실행 내용);
```

비동기 통신 요청 url

DB CRUD 타입에 따라서 선택
C-> post(url, 전달할 데이터)
U -> put(url, 전달할 데이터)

R -> get(url)

D -> delete(url) url파라미터, 쿼리스트링 사용 가능!

비동기통신 | axios를 이용한 서버와의 데이터 통신 - get(), post()

axios.get() 함수를 사용하면 서버로부터 데이터를 받아 올 수 있다.

```
useEffect(() => {  
  axios.get('/api/age')  
    .then(() => {})  
    .catch(() => {});  
}, []);
```

- get('/api/age') 에서 '/api/age'는 spring 서버의 접근 url(주소)이다.
- then()와 catch() 함수 안은 화살표 함수로 실행 내용을 작성한다.
- then()에는 데이터 통신 성공 후 실행 내용을 작성한다.
- catch()에는 데이터 통신 중 오류 발생 시 실행 코드를 작성한다.

axios.post() 함수를 사용하면 서버로 데이터를 전달 할 수 있다.

```
useEffect(() => {  
  axios.post('/api/name', {name : 'kim'})  
    .then((res) => {  
      console.log(res);  
    })  
    .catch((error) => {  
      console.log(error);  
    });  
});
```

- axios.post()의 첫번째 인자로 get()함수와 마찬가지로 접근할 서버 주소를 넣는다.
- axios.post()는 get()함수와 다르게 두번째 인자로 서버에 전달할 데이터를 작성한다.
- 서버에 전달할 데이터는 객체 타입으로 작성한다.
- 객체 타입으로 데이터를 전달하기 때문에 한 번에 여러 데이터를 서버로 전달할 수 있다.
- then()과 catch()의 모든 내용은 get()과 동일하다.
- axios.post()는 서버로 데이터를 전달하는 것이기 때문에 then()에서는 데이터를 받는 코드 보다는, 데이터 전달 결과에 대한 코드를 작성한다.

비동기통신 | axios를 이용한 서버와의 데이터 통신 - get()

```
useEffect(() => {  
  axios.get('/api/age')  
    .then((res) => {  
      console.log(res);  
    })  
    .catch((error) => {  
      console.log(error);  
    });  
}, []);
```

▶ {data: 20, status: 200, statusText: 'OK', headers: AxiosHeaders, config: {...}, ...}

- then() 에 작성하는 화살표 함수는 매개변수를 하나 갖는다.
- 이 매개변수를 출력해보면 응답에 대한 다양한 정보를 객체 타입으로 갖고 있음을 확인할 수 있다.
- 이 매개변수는 함수의 매개변수명이기 때문에 아무렇게나 작성할 수 있지만 응답정보를 갖고 있기 때문에 response의 줄임말 res를 많이 사용한다.
- 현재 우리에게 중요한 것은 응답에 대한 여러 정보 중 실제 서버로부터 받은 데이터이며, 이 데이터는 응답 정보 중 키 값이 data인 곳에 담겨 온다.
- 결론적으로 서버로부터 받은 데이터는 res.data를 통해 접근 가능하다.

▶ AxiosError {message: 'Request failed with status code 404', name: 'AxiosError', code: 'ERR_BAD_REQUEST', config: {...}, request: XMLHttpRequest, ...}

- catch() 에 작성하는 화살표 함수는 매개변수를 하나 갖는다.
- 이 매개변수를 출력해보면 응답실패에 대한 다양한 정보를 객체 타입으로 갖고 있음을 확인할 수 있다.
- 이 매개변수는 함수의 매개변수명이기 때문에 아무렇게나 작성할 수 있지만 통신실패정보를 갖고 있기 때문에 error를 많이 사용한다.
- 여러 오류 데이터 중 키가 message인 정보가 오류에 대한 이유를 간단히 설명한 것이다.
- 404는 요청 경로를 찾을 수 없거나, 페이지명이 잘못되었을 때 발생하는 오류 코드이다.

구분	방법	React axios 요청 예제	@ResponseBody Spring Controller에서 데이터 받기
조회 및 삭제	데이터를 전달하지 않음	조회 : axios.get('/url') 삭제 : axios.delete('/url')	@GetMapping("/url") //삭제는 @DeleteMapping 사용 public void method()
	URL Parameter 방식	조회 : axios.get('/url/3') 삭제 : axios.delete('/url/3')	//(@PathVariable 어노테이션을 사용하여 URL Parameter데이터를 받는다. @GetMapping("/url/{num}") //삭제는 @DeleteMapping 사용 public void method(@PathVariable("num") int num)
	Query String 방식	조회 axios.get('/url?id=hong&age=20') 삭제 axios.delete('/url?id=hong&age=20')	//RequestParam 어노테이션을 이용한 데이터 받기 @GetMapping("/url") //삭제는 @DeleteMapping 사용 public void method(@RequestParam("id") String id, (@RequestParam("age") int age) //넘어오는 데이터의 이름과 일치하는 변수를 가진 DTO 객체로 데이터를 받는다 @GetMapping("/url") //삭제는 @DeleteMapping 사용 public void method(DTO dto)
	get(), delete() 함수의 두번째 매개변수 사용	axios.get('/url', {params:{id:'kim', age:20}}) 삭제는 위 코드에서 get만 delete로 변경	//RequestParam 어노테이션을 이용한 데이터 받기 @GetMapping("/url") //삭제는 @DeleteMapping 사용 public void method(@RequestParam("id") String id, (@RequestParam("age") int age) //넘어오는 데이터의 이름과 일치하는 변수를 가진 DTO 객체로 데이터를 받는다 @GetMapping("/url") //삭제는 @DeleteMapping 사용 public void method(DTO dto)
등록 및 수정	post(), put() 함수의 두번째 매개변수 사용	등록 axios.post('/url', {id:'kim', age:20}) 수정 axios.post('/url', {id:'kim', age:20})	//RequestBody 어노테이션을 이용한 데이터 받기(생략가능) //넘어오는 데이터의 이름과 일치하는 변수를 가진 DTO 객체로 데이터를 받는다 @PostMapping("/url") //수정은 @PutMapping 사용 public void method(@RequestBody DTO dto) putMapping //RequestParam 어노테이션을 이용한 데이터 받기 @GetMapping("/url") //삭제는 @DeleteMapping 사용 public void method(@RequestParam("id") String id, (@RequestParam("age") int age)